Estácio

CAMPUS: Polo Jacareí - SP – Centro

CURSO: Desenvolvimento FullStack

DISCIPLINA: Tecnologias para desenvolvimento de soluções de big data

TURMA: 2026.1

SEMESTRE LETIVO: 2026

ALUNO: Maurício Pereira Campos

MATRÍCULA: 202403843447

Trabalho Prático | DGT2823 Tecnologias para desenvolvimento de soluções de big data:

Objetivos da prática:

1. Descrever como ler um arquivo CSV usando a biblioteca Pandas (Python);

2. Descrever como criar um subconjunto de dados a partir de um conjunto existente usando a biblioteca Pandas (Python);

3. Descrever como configurar o número máximo de linhas a serem exibidas na visualização de um conjunto de dados usando a biblioteca Pandas (Python);

4. Descrever como exibir as primeiras e últimas “N” linhas de um conjunto de dados usando a biblioteca Pandas (Python); Descrever como exibir informações gerais sobre as colunas, linhas e dados de um conjunto de dados usando a biblioteca Pandas (Python);

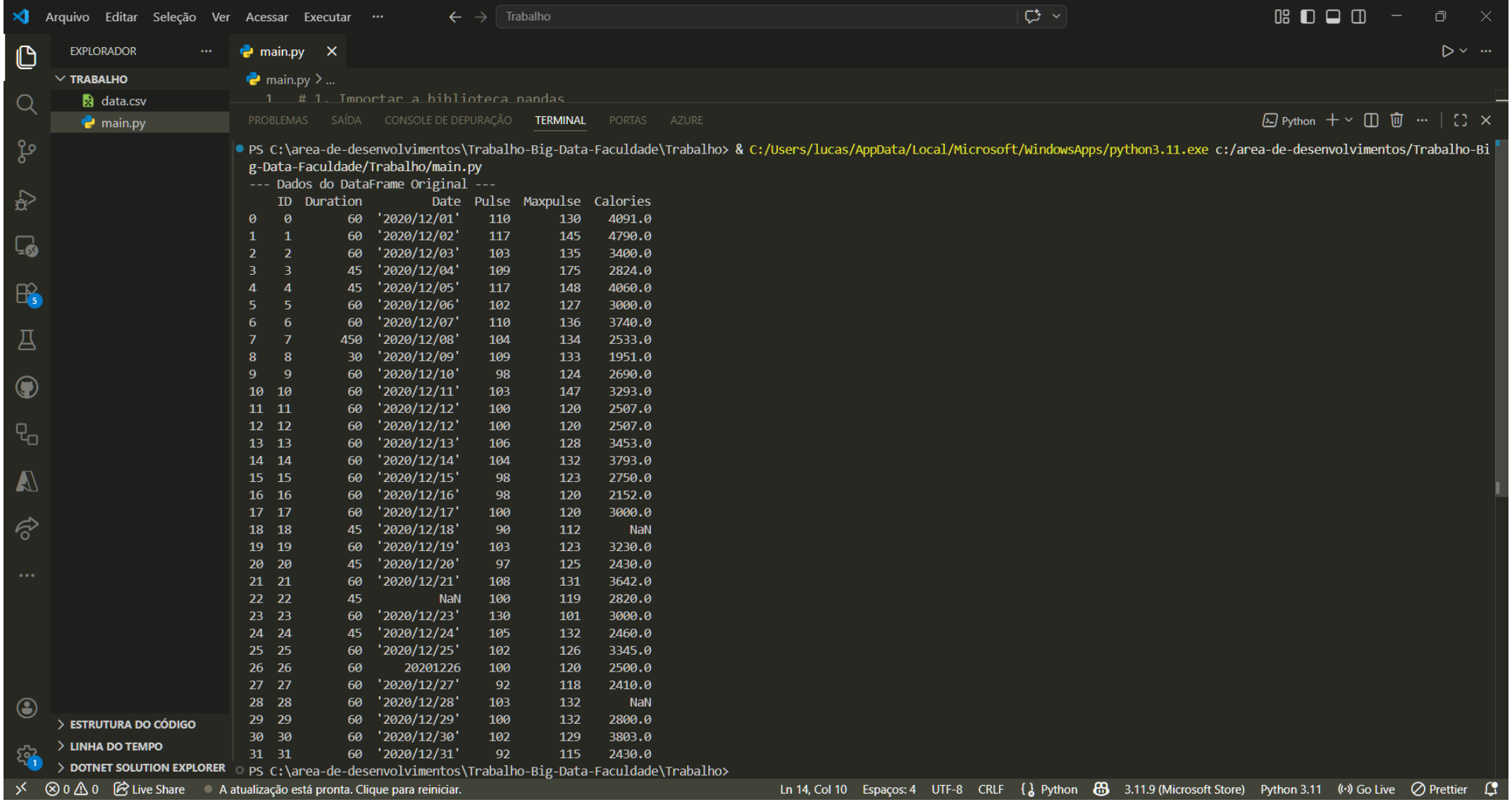
1º Procedimento | Criando o arquivo data.csv

ID	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/0	110	130	4091
1	60	'2020/12/0	117	145	4790
2	60	'2020/12/0	103	135	3400
3	45	'2020/12/0	109	175	2824
4	45	'2020/12/0	117	148	4060
5	60	'2020/12/0	102	127	3000
6	60	'2020/12/0	110	136	3740
7	450	'2020/12/0	104	134	2533
8	30	'2020/12/0	109	133	1951
9	60	'2020/12/1	98	124	2690
10	60	'2020/12/1	103	147	3293
11	60	'2020/12/1	100	120	2507
12	60	'2020/12/1	100	120	2507
13	60	'2020/12/1	106	128	3453
14	60	'2020/12/1	104	132	3793
15	60	'2020/12/1	98	123	2750
16	60	'2020/12/1	98	120	2152
17	60	'2020/12/1	100	120	3000
18	45	'2020/12/1	90	112	NaN
19	60	'2020/12/1	103	123	3230
20	45	'2020/12/2	97	125	2430
21	60	'2020/12/2	108	131	3642
22	45	NaN	100	119	2820
23	60	'2020/12/2	130	101	3000
24	45	'2020/12/2	105	132	2460
25	60	'2020/12/2	102	126	3345
26	60	20201226	100	120	2500
27	60	'2020/12/2	92	118	2410
28	60	'2020/12/2	103	132	NaN
29	60	'2020/12/2	100	132	2800
30	60	'2020/12/3	102	129	3803
31	60	'2020/12/3	92	115	2430

Microatividade 1| Ler um arquivo CSV usando Pandas

Códigos usados neste roteiro

```
# 1. Importar a biblioteca pandas
import pandas as pd

# 2. Criar uma variável e ler o conteúdo do arquivo CSV
# Parâmetros solicitados:
# sep=';' (o arquivo usa ponto e vírgula)
# engine='python' (garante compatibilidade com alguns formatos de CSV)
# encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)

df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

# 3. Exibir os dados na tela
print("--- Dados do DataFrame Original ---")
print(df)
```

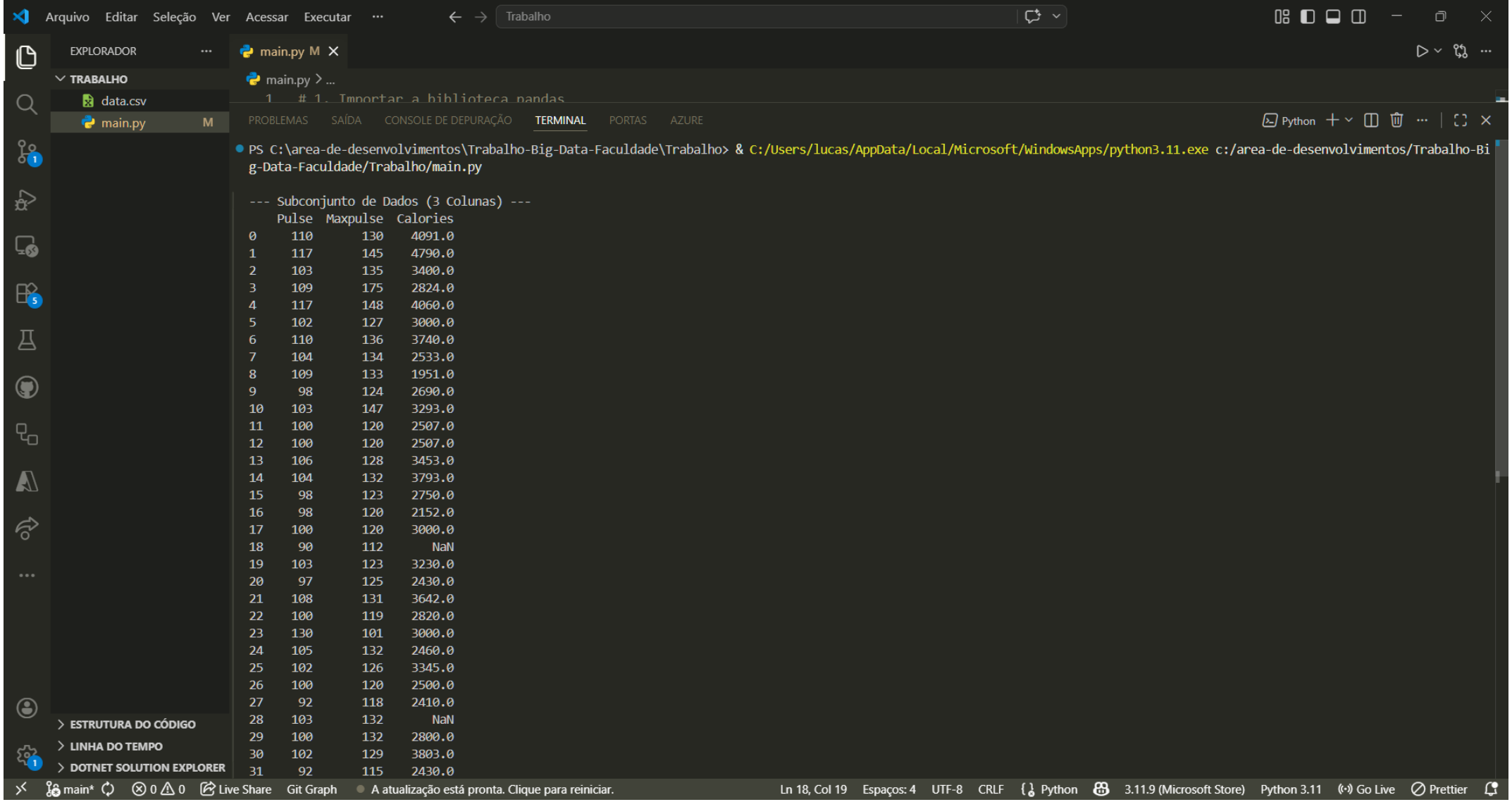


•**import pandas as pd**: É a convenção padrão da comunidade. O pd facilita a chamada dos métodos.
•**df**: Abreviação de *DataFrame*, que é a estrutura de tabela do Pandas.
•**sep=';'**: Por padrão, o Pandas busca por vírgulas. Como seu arquivo usa ;, precisamos avisá-lo, senão ele colocará tudo em uma única coluna.

Microatividade 2 | Criar um subconjunto de dados

Códigos usados neste roteiro

```
# 1. Importar a biblioteca pandas
import pandas as pd


# 2. Criar uma variável e ler o conteúdo do arquivo CSV
# Parâmetros solicitados:
# sep=';' (o arquivo usa ponto e vírgula)
# engine='python' (garante compatibilidade com alguns formatos de CSV)
# encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)


df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 1. Criar uma nova variável para o subconjunto
# Selecionando 3 colunas: 'Pulse', 'Maxpulse' e 'Calories'
df_reduzido = df[['Pulse', 'Maxpulse', 'Calories']]


# 2. Exibir o novo subconjunto
print("\n--- Subconjunto de Dados (3 Colunas) ---")
print(df_reduzido)
```



Explicação técnica:

Ao usar colchetes duplos [[...]], estamos passando uma lista de colunas para o DataFrame original, o que gera um novo DataFrame apenas com essas fatias de dados.

Microatividade 3 | Configurar o número máximo de linhas exibidas

Códigos usados neste roteiro

```
# 1. Importar a biblioteca pandas
import pandas as pd


# 2. Criar uma variável e ler o conteúdo do arquivo CSV
# Parâmetros solicitados:
# sep=';' (o arquivo usa ponto e vírgula)
# engine='python' (garante compatibilidade com alguns formatos de CSV)
# encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)


df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 1. Definir o valor máximo de linhas para exibição como 9999
pd.options.display.max_rows = 9999


# 2. Imprimir o conjunto original usando o método to_string()
# O to_string() transforma o DataFrame em uma string completa, sem resumos.
print("\n--- Visualização Completa (Microatividade 3) ---")
print(df.to_string())
```

```
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho> & C:/Users/lucas/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/area-de-desenvolvimentos/Trabalho-Big-Data-Faculdade/Trabalho/main.py

--- Visualização Completa (Microatividade 3) ---
    ID  Duration         Date  Pulse  Maxpulse  Calories
0    0        60  '2020/12/01'    110       130    4091.0
1    1        60  '2020/12/02'    117       145    4790.0
2    2        60  '2020/12/03'    103       135    3400.0
3    3        45  '2020/12/04'    109       175    2824.0
4    4        45  '2020/12/05'    117       148    4060.0
5    5        60  '2020/12/06'    102       127    3000.0
6    6        60  '2020/12/07'    110       136    3740.0
7    7       450  '2020/12/08'    104       134    2533.0
8    8        30  '2020/12/09'    109       133    1951.0
9    9        60  '2020/12/10'     98       124    2690.0
10  10        60  '2020/12/11'    103       147    3293.0
11  11        60  '2020/12/12'    100       120    2507.0
12  12        60  '2020/12/12'    100       120    2507.0
13  13        60  '2020/12/13'    106       128    3453.0
14  14        60  '2020/12/14'    104       132    3793.0
15  15        60  '2020/12/15'     98       123    2750.0
16  16        60  '2020/12/16'     98       120    2152.0
17  17        60  '2020/12/17'    100       120    3000.0
18  18        45  '2020/12/18'     90       112       NaN
19  19        60  '2020/12/19'    103       123    3230.0
20  20        45  '2020/12/20'     97       125    2430.0
21  21        60  '2020/12/21'    108       131    3642.0
22  22        45           NaN    100       119    2820.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       NaN
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho>
```

Explicação técnica:

Por padrão, se um DataFrame for muito grande, o Pandas "esconde" as linhas do meio para não poluir a tela. Aqui, vamos forçá-lo a mostrar tudo (se necessário) usando as opções de configuração.

Microatividade 4 | Exibir as primeiras e últimas “N” linhas

Códigos usados neste roteiro

```
# 1. Importar a biblioteca pandas
import pandas as pd


# 2. Criar uma variável e ler o conteúdo do arquivo CSV
# Parâmetros solicitados:
# sep=';' (o arquivo usa ponto e vírgula)
# engine='python' (garante compatibilidade com alguns formatos de CSV)
# encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)


df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 1. Definir o valor máximo de linhas para exibição como 9999
pd.options.display.max_rows = 9999


# 2. Imprimir o conjunto original usando o método to_string()
# O to_string() transforma o DataFrame em uma string completa, sem resumos.
print("\n--- Visualização Completa (Microatividade 3) ---")
print(df.to_string())
```

```
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho> & C:/Users/lucas/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/area-de-desenvolvimentos/Trabalho-Big-Data-Faculdade/Trabalho/main.py

--- Primeiras 10 linhas ---
   ID  Duration         Date  Pulse  Maxpulse  Calories
0   0        60  '2020/12/01'    110       130    4091.0
1   1        60  '2020/12/02'    117       145    4790.0
2   2        60  '2020/12/03'    103       135    3400.0
3   3        45  '2020/12/04'    109       175    2824.0
4   4        45  '2020/12/05'    117       148    4060.0
5   5        60  '2020/12/06'    102       127    3000.0
6   6        60  '2020/12/07'    110       136    3740.0
7   7       450  '2020/12/08'    104       134    2533.0
8   8        30  '2020/12/09'    109       133    1951.0
9   9        60  '2020/12/10'     98       124    2690.0

--- Últimas 10 linhas ---
    ID  Duration          Date  Pulse  Maxpulse  Calories
22  22        45           NaN    100       119    2820.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       NaN
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho>
```

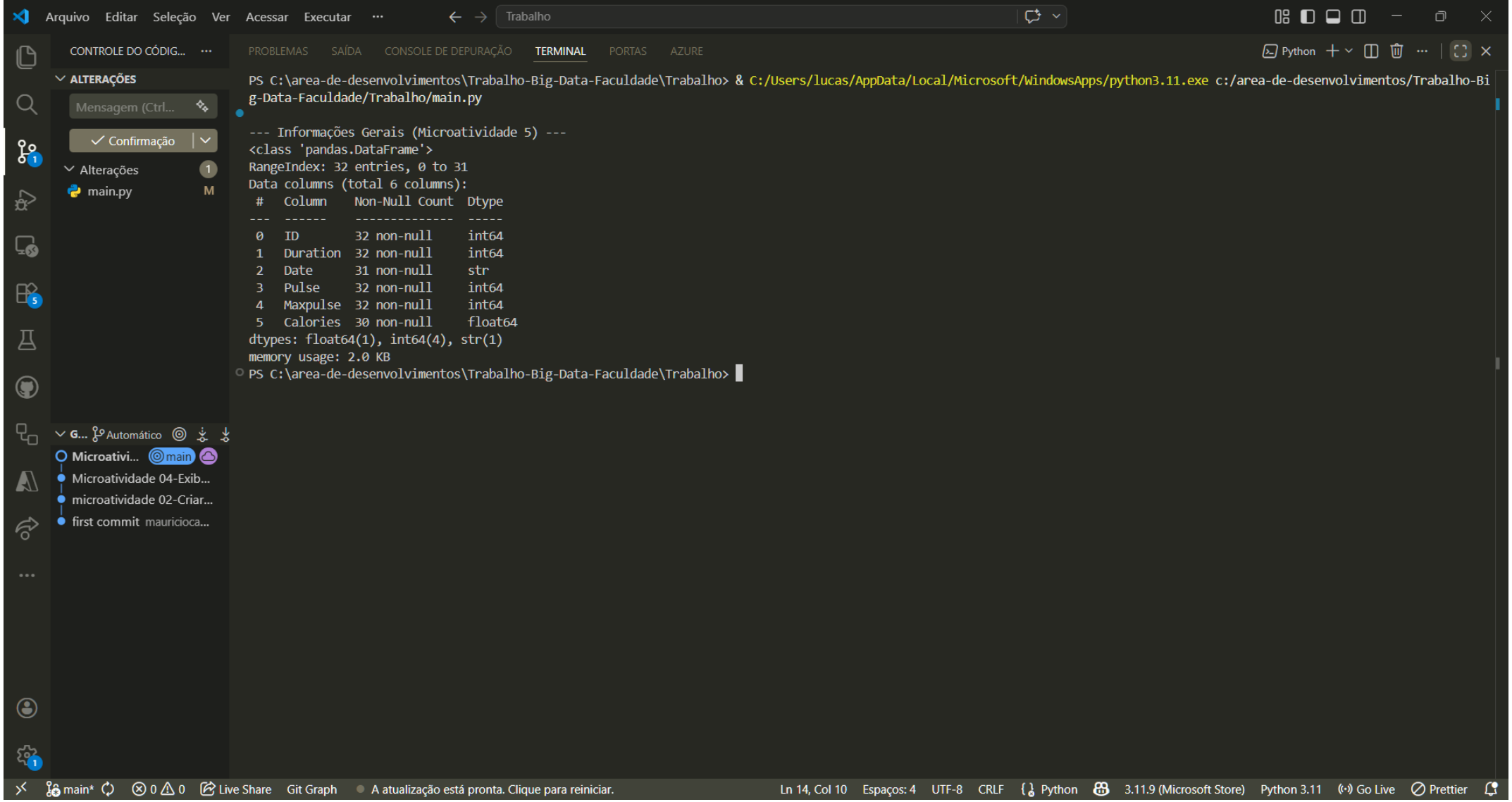
Explicação técnica:

Em Big Data, raramente olhamos o arquivo inteiro. Geralmente, usamos o head (cabeça) e o tail (cauda) para verificar se o início e o fim dos dados fazem sentido.

Microatividade 5 | Informações gerais sobre o conjunto de dados

Códigos usados neste roteiro

```
# 1. Importar a biblioteca pandas
import pandas as pd


# 2. Criar uma variável e ler o conteúdo do arquivo CSV
# Parâmetros solicitados:
# sep=';' (o arquivo usa ponto e vírgula)
# engine='python' (garante compatibilidade com alguns formatos de CSV)
# encoding='ISO-8859-1' ou 'utf-8' (opcional, vamos usar utf-8 por padrão)


df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 1. Imprimir informações gerais do DataFrame
print("\n--- Informações Gerais (Microatividade 5) ---")
df.info()
```



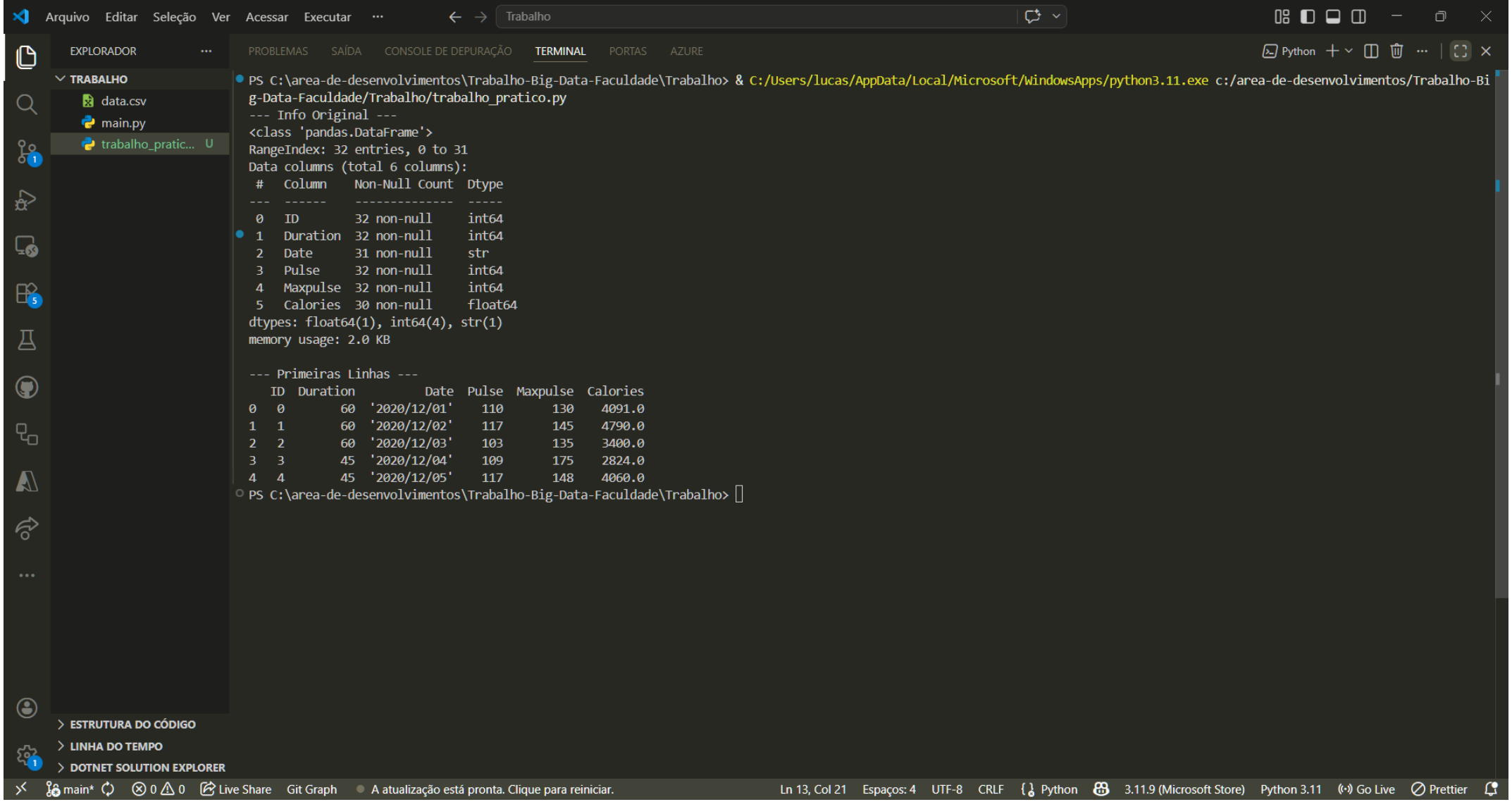
Explicação técnica:

Esta é a etapa mais importante para um Analista de Dados. O método .info() é o "raio-x" do arquivo: total de linhas, colunas, nulos e memória.

Trabalho Prático | Passo 1 ao 5: Preparação e Cópia de Segurança

Códigos usados neste roteiro

```
import pandas as pd

# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

# 2. Verificar a importação
print("--- Info Original ---")
df.info()
print("\n--- Primeiras Linhas ---")
print(df.head())

# 3. Criar uma variável com a CÓPIA dos dados
df_limpo = df.copy()
```



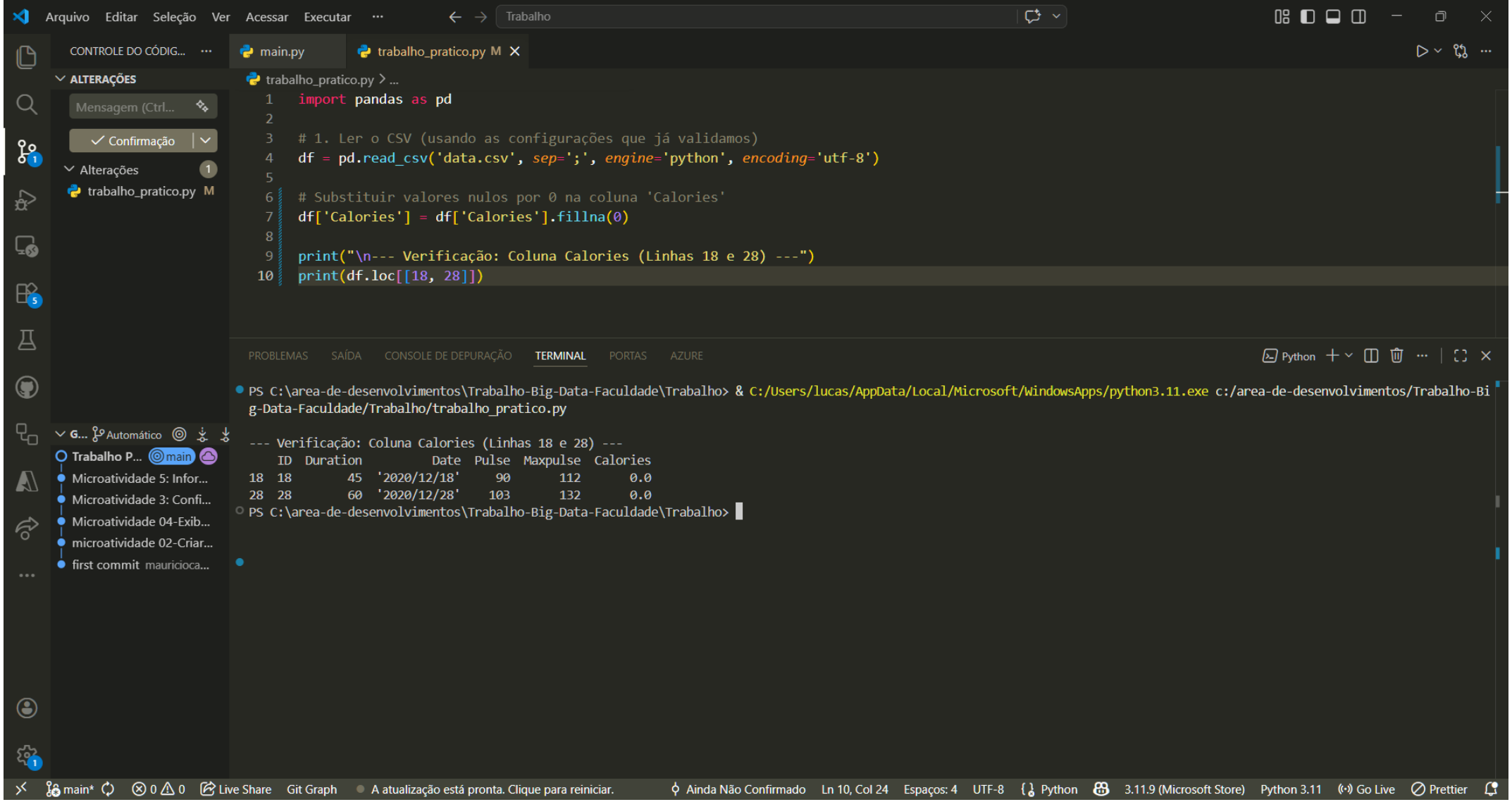
Explicação técnica:

Primeiro, carregamos os dados e criamos uma cópia. Trabalhar em uma cópia é uma boa prática para nunca perder os dados originais se algo der errado na limpeza.

Trabalho Prático| Passo 6: Tratando a coluna 'Calories

Códigos usados neste roteiro

```
import pandas as pd


# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# Substituir valores nulos por 0 na coluna 'Calories'
df['Calories'] = df['Calories'].fillna(0)


print("\n--- Verificação: Coluna Calories (Linhas 18 e 28) ---")
print(df.loc[[18, 28]])
```



Explicação técnica:

As linhas **18** e **28** possuem valores nulos (NaN). Vamos assumir que, para este estudo, onde não há registro de calorias, o valor será zero.

Trabalho Prático | Passo 7: O desafio da coluna 'Date'

Códigos usados neste roteiro

```
import pandas as pd


# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 1. Substituir nulos por '1900/01/01'
df['Date'] = df['Date'].fillna('1900/01/01')


print("\n--- Verificação: Coluna Date com 1900/01/01 ---")
print(df.loc[[22]]) # A linha 22 era a que tinha NaN


# 2. Tentar transformar em datetime (ISSO VAI GERAR ERRO)
try:
    df['Date'] = pd.to_datetime(df['Date'], format='%Y/%m/%d')
except Exception as e:
    print(f"\n❌ ERRO ESPERADO ENCONTRADO: {e}")
```

```python
import pandas as pd

# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')

# 1. Substituir nulos por '1900/01/01'
df['Date'] = df['Date'].fillna('1900/01/01')

print("\n--- Verificação: Coluna Date com 1900/01/01 ---")
print(df.loc[[22]]) # A linha 22 era a que tinha NaN

# 2. Tentar transformar em datetime (ISSO VAI GERAR ERRO)
try:
    df['Date'] = pd.to_datetime(df['Date'], format='%Y/%m/%d')
except Exception as e:
    print(f"\n❌ ERRO ESPERADO ENCONTRADO: {e}")
```

```
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho> & C:/Users/lucas/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/area-de-desenvolvimentos/Trabalho-Big-Data-Faculdade/Trabalho/trabalho_pratico.py

--- Verificação: Coluna Date com 1900/01/01 ---
    ID  Duration        Date  Pulse  Maxpulse  Calories
22  22        45  1900/01/01    100       119    2820.0

❌ ERRO ESPERADO ENCONTRADO: time data "'2020/12/01'" doesn't match format "%Y/%m/%d". You might want to try:
    - passing `format` if your strings have a consistent format;
    - passing `format='ISO8601'` if your strings are all ISO8601 but not necessarily in exactly the same format;
    - passing `format='mixed'`, and the format will be inferred for each element individually. You might want to use `dayfirst` alongside this.
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho>
```

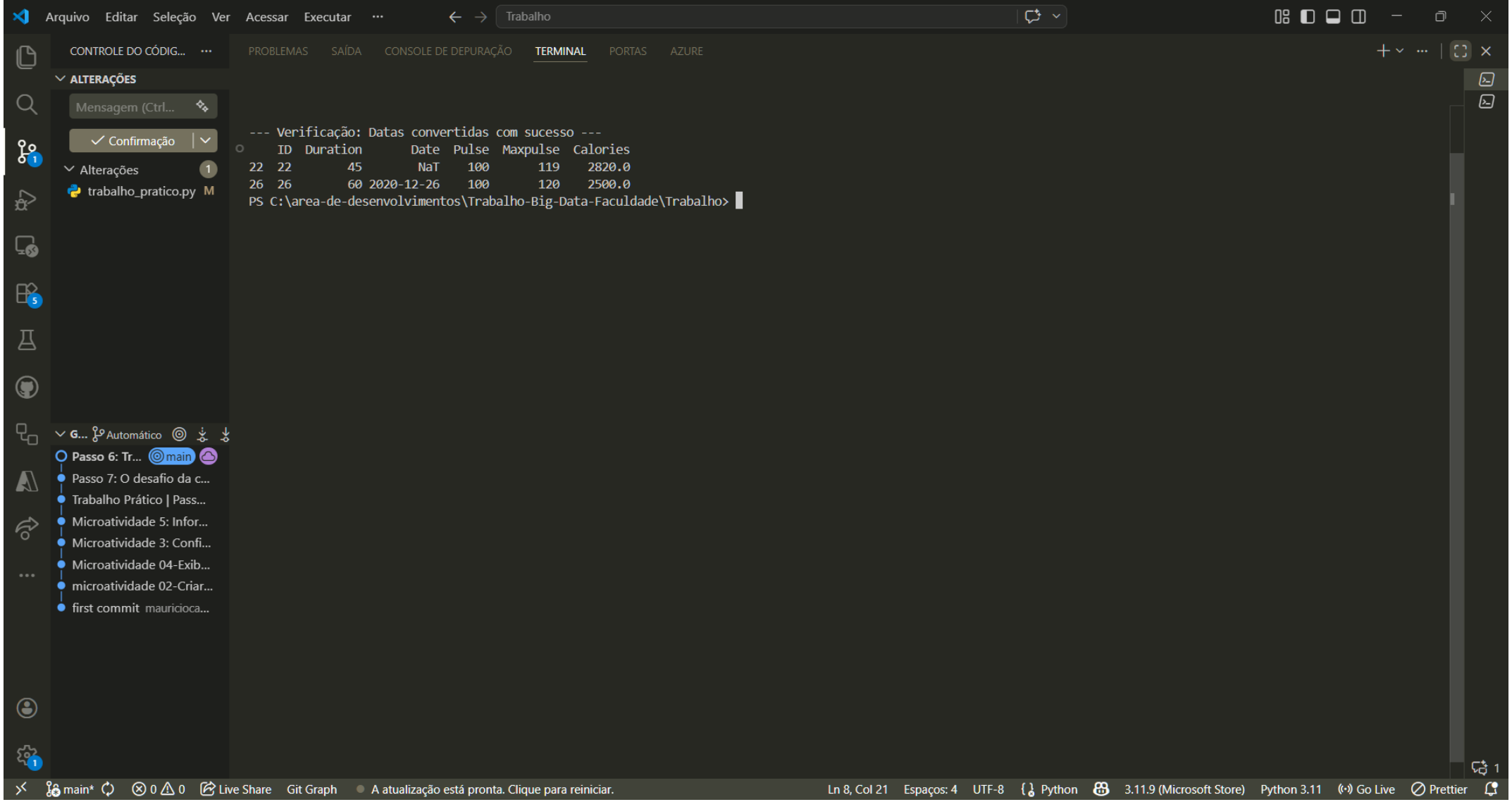
Explicação técnica:

Por que deu erro? > O Pandas é rigoroso. Se você notar o arquivo original, as datas estão entre aspas simples (ex: '2020/12/01'). Quando inserimos 1900/01/01 sem as aspas ou com um formato que ele não espera dentro daquela mistura de strings, ele trava para garantir a integridade. Além disso, temos o registro 20201226 (sem barras) na linha 26 que também vai "quebrar" a conversão automática.

Trabalho Prático | Passo 8 ao 11: Corrigindo as Datas e Formatos

Códigos usados neste roteiro

```
import pandas as pd
import numpy as np


# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 8. Substituir o valor '1900/01/01' por NaN (Not a Number)
df_limpo = df.copy()
df_limpo['Date'] = (
    df_limpo['Date']
    .astype('string')
    .str.strip()
    .str.strip("'")
    .str.strip('"')
)


df_limpo['Date'] = df_limpo['Date'].replace('1900/01/01', pd.NA)


# 9. Corrigir o erro do valor "20201226" (Linha 26)
# Primeiro usamos o replace para ajustar a string e depois tentamos a conversão
df_limpo['Date'] = df_limpo['Date'].replace('20201226', '2020/12/26')


# 10. Agora convertemos toda a coluna para o formato Datetime
# O parâmetro errors='coerce' transformaria erros em NaN,
# mas como o roteiro pede para tratar especificamente, usamos a conversão direta:
df_limpo['Date'] = pd.to_datetime(df_limpo['Date'], format='%Y/%m/%d', errors='coerce')


print("\n--- Verificação: Datas convertidas com sucesso ---")
print(df_limpo.loc[[22, 26]]) # Verificando a linha 22 (que virou NaN) e a 26 (ajustada)
```



Explicação técnica:

Tratar o erro do valor 1900/01/01 e o formato numérico da linha 26 (20201226).

Trabalho Prático | Passo 12: Removendo registros nulos

Códigos usados neste roteiro

```
import pandas as pd
import numpy as np


# 1. Ler o CSV (usando as configurações que já validamos)
df = pd.read_csv('data.csv', sep=';', engine='python', encoding='utf-8')


# 11. Remover registros onde a coluna 'Date' possui valores nulos
# O subset garante que ele olhe apenas para a coluna Date para decidir a exclusão
df_limpo = df.copy()
df_limpo.dropna(subset=['Date'], inplace=True)


# 12. Imprimir o resultado final para validação
print("\n--- DataFrame Final após Limpeza ---")
print(df_limpo.to_string())


# Verificação final de informações
print("\n--- Resumo Final ---")
df_limpo.info()
```

```
PS C:\area-de-desenvolvimentos\Trabalho-Big-Data-Faculdade\Trabalho> & C:/Users/lucas/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/area-de-desenvolvimentos/Trabalho-Big-Data-Faculdade/Trabalho/

--- DataFrame Final após Limpeza ---
    ID  Duration          Date  Pulse  Maxpulse  Calories
0    0        60  '2020/12/01'    110       130    4091.0
1    1        60  '2020/12/02'    117       145    4790.0
2    2        60  '2020/12/03'    103       135    3400.0
3    3        45  '2020/12/04'    109       175    2824.0
4    4        45  '2020/12/05'    117       148    4060.0
5    5        60  '2020/12/06'    102       127    3000.0
6    6        60  '2020/12/07'    110       136    3740.0
7    7       450  '2020/12/08'    104       134    2533.0
8    8        30  '2020/12/09'    109       133    1951.0
9    9        60  '2020/12/10'     98       124    2690.0
10  10        60  '2020/12/11'    103       147    3293.0
11  11        60  '2020/12/12'    100       120    2507.0
12  12        60  '2020/12/12'    100       120    2507.0
13  13        60  '2020/12/13'    106       128    3453.0
14  14        60  '2020/12/14'    104       132    3793.0
15  15        60  '2020/12/15'     98       123    2750.0
16  16        60  '2020/12/16'     98       120    2152.0
17  17        60  '2020/12/17'    100       120    3000.0
18  18        45  '2020/12/18'     90       112       NaN
19  19        60  '2020/12/19'    103       123    3230.0
20  20        45  '2020/12/20'     97       125    2430.0
21  21        60  '2020/12/21'    108       131    3642.0
23  23        60  '2020/12/23'    130       101    3000.0
24  24        45  '2020/12/24'    105       132    2460.0
25  25        60  '2020/12/25'    102       126    3345.0
26  26        60      20201226    100       120    2500.0
27  27        60  '2020/12/27'     92       118    2410.0
28  28        60  '2020/12/28'    103       132       NaN
29  29        60  '2020/12/29'    100       132    2800.0
30  30        60  '2020/12/30'    102       129    3803.0
31  31        60  '2020/12/31'     92       115    2430.0

--- Resumo Final ---
<class 'pandas.DataFrame'>
Index: 31 entries, 0 to 31
Data columns (total 6 columns):
 #   Column    Non-Null Count  Dtype
---  ------    --------------  -----
 0   ID        31 non-null     int64
 1   Duration  31 non-null     int64
 2   Date      31 non-null     str
 3   Pulse     31 non-null     int64
 4   Maxpulse  31 non-null     int64
 5   Calories  29 non-null     float64
dtypes: float64(1), int64(4), str(1)
memory usage: 2.1 KB
```

Explicação técnica:

O objetivo final de uma limpeza muitas vezes é descartar o que sobrou de "lixo" ou dados incompletos que podem enviesar a análise. Como a linha 22 não tem data válida, vou removê-la.

Documentação (passo a passo) - baseado nos commits do repositório

Abaixo está um texto de documentação **derivado dos commits** do repo mauriciocampos1234/Trabalho-Big-Data-Pandas-Faculdade (branch main), todos feitos em **2026-04-10**.

1) Commit inicial
Commit: b2c8176 - *"first commit"*
O que foi feito:
Criação inicial do repositório e inclusão dos primeiros arquivos do projeto.

2) Microatividade 02 - Criar um subconjunto de dados
Commit: 4d05f4b - *"microatividade 02-Criar um subconjunto de dados"*
O que foi feito (passos):
Carregar o dataset (provavelmente com pandas).
Selecionar um recorte do conjunto de dados (ex.: algumas colunas e/ou algumas linhas).
Armazenar esse subconjunto em uma variável/estrutura para análises posteriores.
Validar o subconjunto exibindo amostras (ex.: head()).

3) Microatividade 03 - Configurar o número máximo de linhas exibidas
Commit: 40d852b - *"Microatividade 3: Configurar o número máximo de linhas exibidas"*
O que foi feito (passos):
Ajustar configurações de exibição do pandas (ex.: pd.set_option).
Definir um limite maior/adequado de linhas para inspeção.
Confirmar que as impressões/prints do dataframe ficaram mais informativas.

Documentação (passo a passo) - baseado nos commits do repositório

4) Microatividade 04 - Exibir as primeiras e últimas “N” linhas
Commit: 7e640a7 - *“Microatividade 04-Exibir as primeiras e últimas "N" linhas”*
O que foi feito (passos):
Definir um valor **N** para visualização.
Exibir as primeiras N linhas do dataset (ex.: df.head(N)).
Exibir as últimas N linhas do dataset (ex.: df.tail(N)).
Usar isso para checar rapidamente a estrutura e consistência dos dados.

5) Microatividade 05 - Informações gerais sobre o conjunto de dados
Commit: 4e135dd - *“Microatividade 5: Informações gerais sobre o conjunto de dados”*
O que foi feito (passos):
Obter visão geral do dataframe (linhas, colunas, tipos).
Verificar tipos de dados e presença de valores ausentes.
Consultar estatísticas/resumo (ex.: describe() quando aplicável).
Identificar possíveis colunas problemáticas para tratamento (ex.: datas, numéricos como texto, nulos).

Documentação (passo a passo) - baseado nos commits do repositório

5) Trabalho Prático - Passo 1 ao 5: Preparação e Cópia de Segurança
Commit: 852dc57 - *“Trabalho Prático | Passo 1 ao 5: Preparação e Cópia de Segurança”*
O que foi feito (passos):
Preparar o ambiente (imports e configurações iniciais).
Carregar o dataset original.
Criar uma **cópia de segurança** do dataframe (para preservar o original).
Padronizar/organizar estrutura inicial (nomes de colunas, ordenação, etc., se aplicável).
Fazer validações iniciais para garantir que a base está pronta para limpeza.

6) Passo 6 - Tratando a coluna Calories
Commit: c9b13ab - *“Passo 6: Tratando a coluna 'Calories'”*
O que foi feito (passos):
Inspecionar a coluna Calories (tipo atual e valores inconsistentes).
Converter para tipo numérico quando necessário.
Tratar valores inválidos (ex.: textos, símbolos, vazios) com conversão segura.
Verificar se surgiram NaN após conversão e preparar para limpeza posterior.

Documentação (passo a passo) - baseado nos commits do repositório

7) Passo 7 - O desafio da coluna Date (Parte 1)
Commit: 05a9481 - *“Passo 7: O desafio da coluna 'Date' (Parte 1)”*
O que foi feito (passos):
Diagnosticar o formato da coluna Date (strings, formatos mistos, valores inválidos).
Definir estratégia de parsing/conversão (ex.: to_datetime).
Executar uma primeira tentativa de conversão e observar erros/inconsistências.
Marcar/identificar quais padrões de data precisam de tratamento adicional.

8) Passos 8 ao 11 — Corrigindo as Datas e Formatos
Commit: 7e0f711 — *“Passo 8 ao 11: Corrigindo as Datas e Formatos”*
O que foi feito (passos):
Padronizar a coluna Date para um tipo de data consistente.
Corrigir formatos conflitantes (ex.: dia/mês vs mês/dia, strings fora do padrão).
Tratar valores inválidos que não puderam ser convertidos (virando NaT/nulos).
Validar a conversão final (amostras, contagem de nulos, checagem de tipo).
Se houver outras colunas com formatação semelhante, aplicar padronização equivalente.

Documentação (passo a passo) - baseado nos commits do repositório

9) Passo 12 - Removendo registros nulos
Commit: ad302a6 - *“Passo 12: Removendo registros nulos”*
O que foi feito (passos):
Identificar registros com valores nulos (NaN/NaT) após os tratamentos.
Definir regra de remoção (ex.: remover linhas com nulos em colunas críticas).
Remover os registros nulos conforme a regra.
Validar resultado: comparar quantidade de linhas antes/depois e garantir consistência.

Link do repositório do GitHub

https://github.com/mauriciocampos1234/Trabalho-Big-Data-Pandas-Faculdade